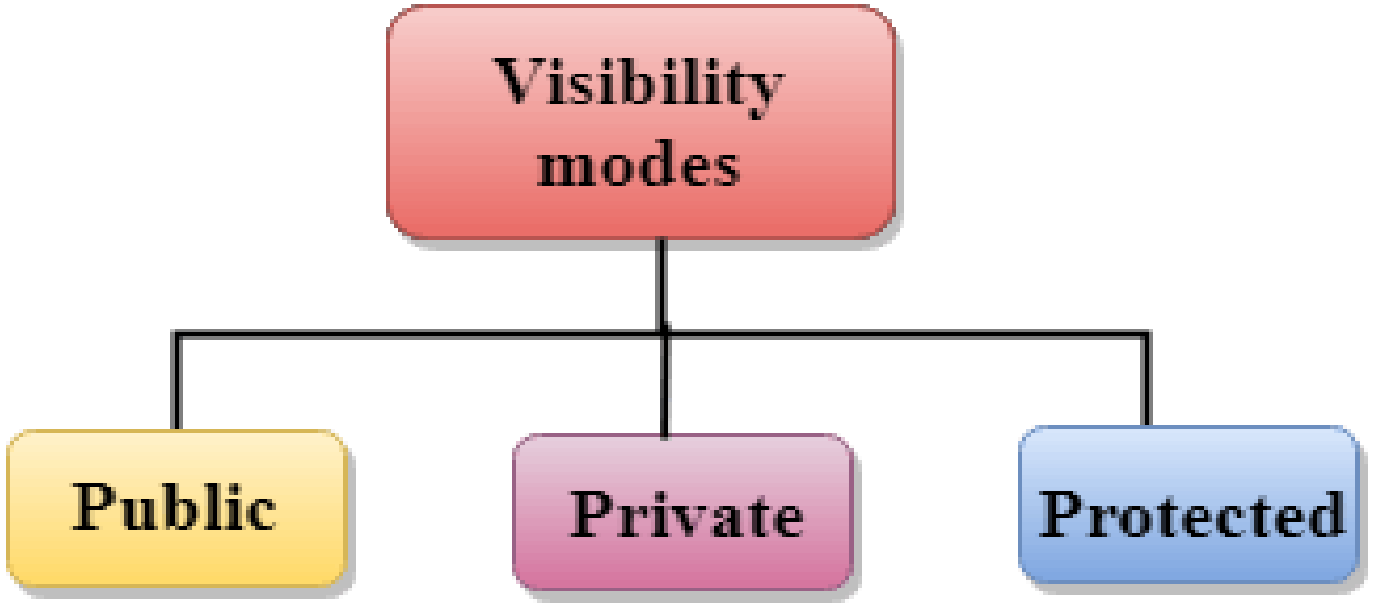
Module 6
Inheritance

Dr. S. Vinila Jinny

Definition

- Inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically.
- In such way, we can reuse, extend or modify the attributes and behaviors which are defined in other class.
- The class which inherits the members of another class is called derived class and the class whose members are inherited is called base class.
- The derived class is the specialized class for the base class.

Visibilty modes



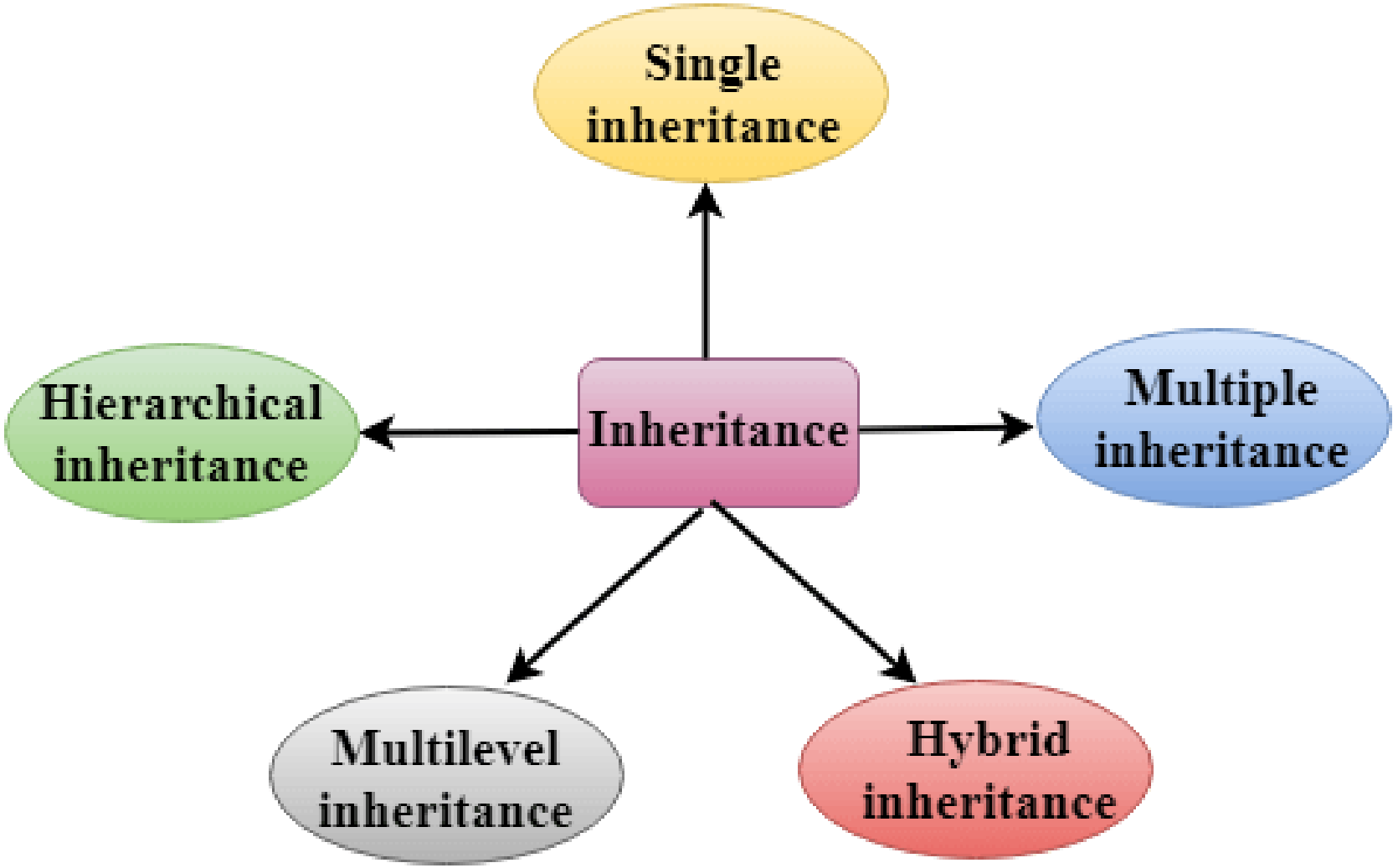
Visibility mode

- Public: When the member is declared as public, it is accessible to all the functions of the program.
- Private: When the member is declared as private, it is accessible within the class only.
- Protected: When the member is declared as protected, it is accessible within its own class as well as the class immediately derived from it.

Visibility of Inherited Members

Base class visibility	Derived class visibility		
	Public	Private	Protected
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

Types of Inheritance



Derived Class

To inherit from a class, use the : symbol.

A Derived class is defined as the class derived from the base class.

Syntax :

```
class derived_class_name : visibility-mode base_class_name
{
    // body of the derived class.
}
```

Where,

visibility mode: The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited. It can be public or private.

- The private members of the base class are never inherited.

Derived Class

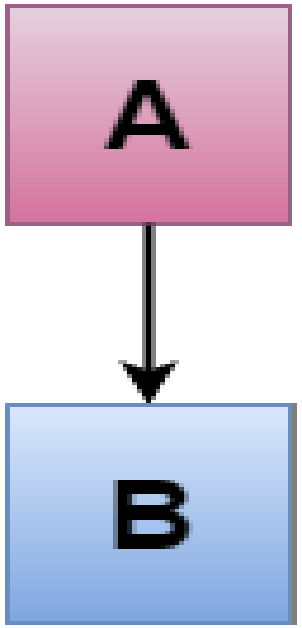
privately inherited : When the base class is **privately inherited** by the derived class, public members of the base class becomes the private members of the derived class. Therefore, the public members of the base class are not accessible by the objects of the derived class only by the member functions of the derived class.

publicly inherited : When the base class is **publicly inherited** by the derived class, public members of the base class also become the public members of the derived class. Therefore, the public members of the base class are accessible by the objects of the derived class as well as by the member functions of the base class.

Restrictions in Inheritance

- A derived class inherits all base class methods with the following exceptions –
- Constructors, destructors and copy constructors of the base class.
- Overloaded operators of the base class.
- The friend functions of the base class.

Single Inheritance



Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.

Single Inheritance

```
#include <iostream>
using namespace std;
class A
{
    int a = 4;
    int b = 5;
    public:
    int mul()
    {
        int c = a*b;
        return c;
    }
};
```

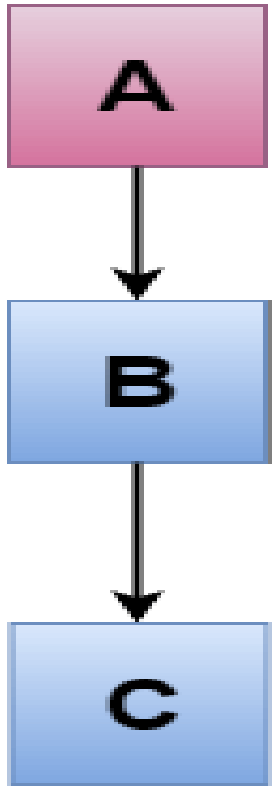
```
class B : private A
{
    public:
    void display()
    {
        int result = mul();
        cout <<"Multiplication of a and b is : "<<result<<endl;
    }
};
int main()
{
   B b;
   b.display();

    return 0;
}
```

Output
Multiplication of a and b is : 20

Multilevel Inheritance

- Multilevel inheritance is a process of deriving a class from another derived class.



Multilevel Inheritance

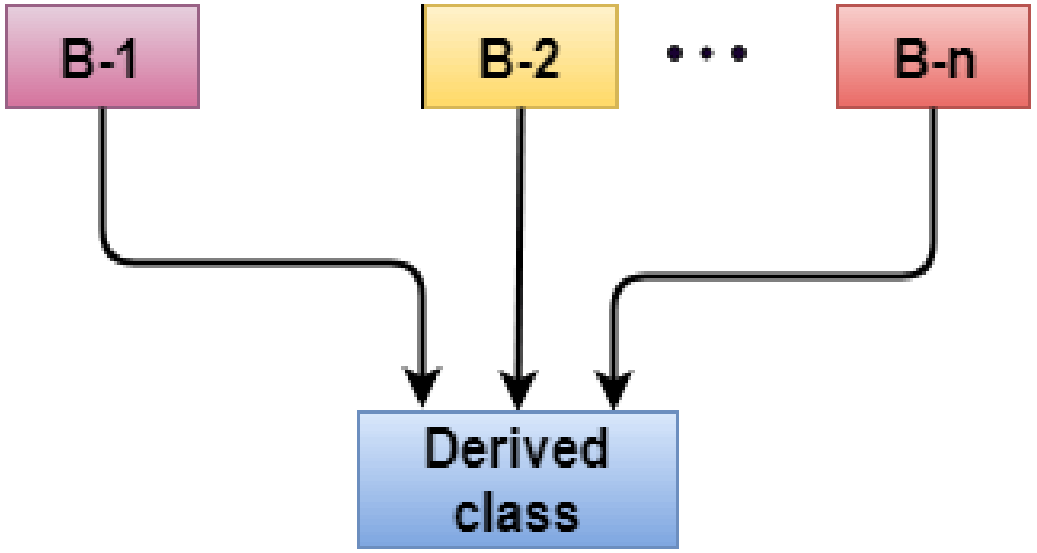
```
#include <iostream>
using namespace std;
 class Animal {
   public:
 void eat() {
    cout<<"Eating..."<<endl;
 }
   };
   class Dog: public Animal
   {
       public:
     void bark(){
    cout<<"Barking..."<<endl;
     }
   };
0;
}
```

```
class BabyDog: public Dog
   {
       public:
     void weep()
 {
    cout<<"Weeping...";
     }
   };
int main(void) {
    BabyDog d1;
    d1.eat();
    d1.bark();
     d1.weep();
     return 0;
}
```

Output:
Eating...
Barking...
Weeping...

Multiple Inheritance

- **Multiple inheritance** is the process of deriving a new class that inherits the attributes from two or more classes.



Multiple Inheritance

```
#include <iostream>
using namespace std;
class A
{
   protected:
    int a;
   public:
   void get_a(int n)
   {
       a = n;
   }
};
```

```
class B
{
   protected:
   int b;
   public:
   void get_b(int n)
   {
      b = n;
   }
};
class C : public A,public B
{
  public:
   void display()
   {
      cout << "The value of a is : " <<a<< std::endl;
      cout << "The value of b is : " <<b<< std::endl;
      cout<<"Addition of a and b is : "<<a+b;
   }
};
```

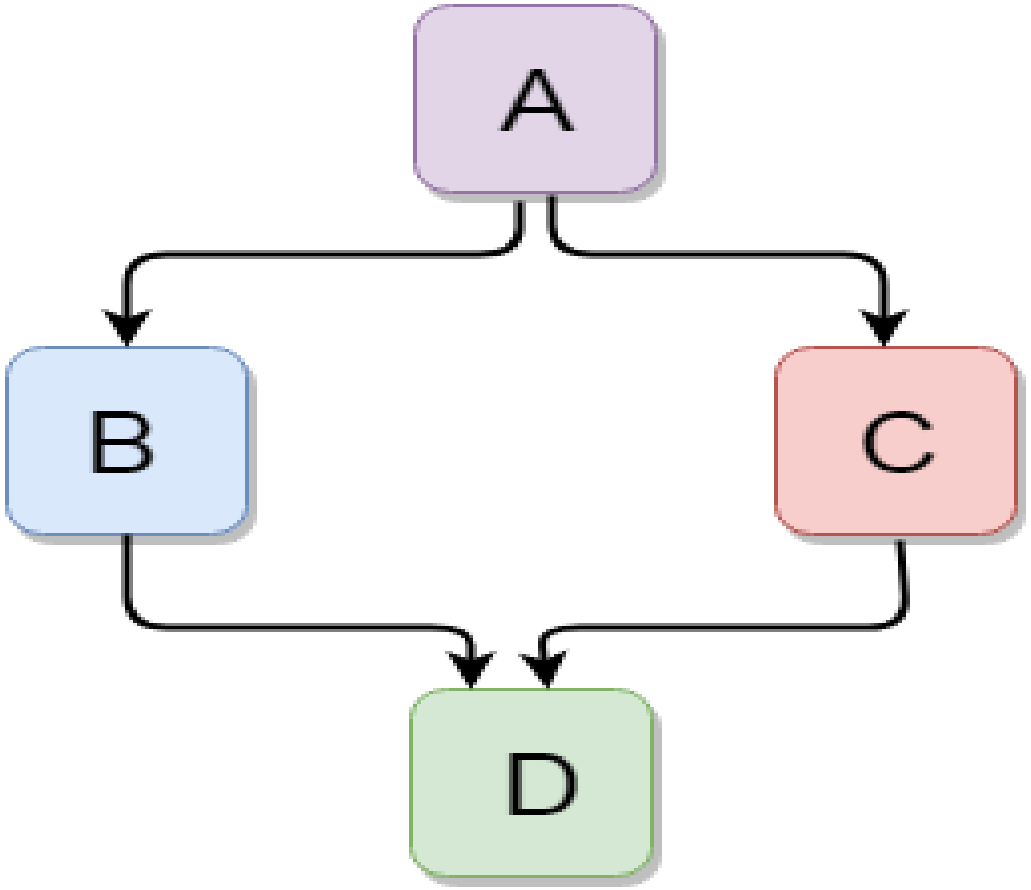
```
int main()
{
   C c;
   c.get_a(10);
   c.get_b(20);
   c.display();

   return 0;
}
```

Output:

```
The value of a is : 10
The value of b is : 20
Addition of a and b is : 30
```

Hybrid Inheritance



Example

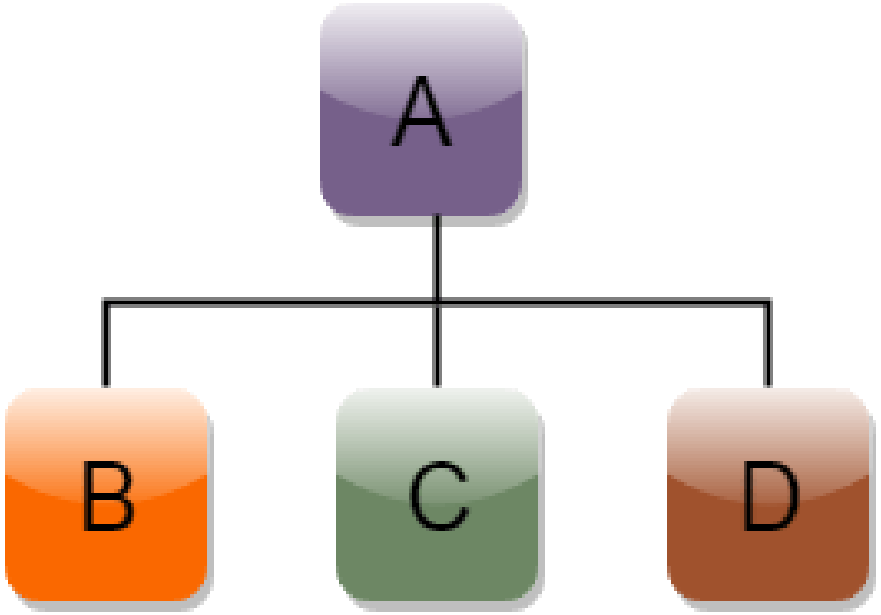
```
#include <iostream>
using namespace std;
class A
{
   protected:
   int a;
   public:
   void get_a()
   {
cout << "Enter the value of 'a' : " <<endl;
      cin>>a;
   }
};
class B : public A
{
   protected:
   int b;
   public:
   void get_b()
   {
 cout << "Enter the value of 'b' : " <<endl;
      cin>>b;
   }
};
class C
{
   protected:
   int c;
   public:
   void get_c()
   {
 cout << "Enter the value of c is : " <<endl;
      cin>>c;
   }
};
class D : public B, public C
{
   protected:
   int d;
   public:
   void mul()
   {
        get_a();
        get_b();
        get_c();
cout << "Multiplication of a,b,c is : " <<a*b*c<<endl;
   }
};
int main()
{
   D d;
   d.mul();
   return 0;
}
```

Output

```
Enter the value of 'a' :
Enter the value of 'b' :
Enter the value of c is :
Multiplication of a,b,c is : 0
```

Hierarchical Inheritance

- Hierarchical inheritance is defined as the process of deriving more than one class from a base class.



Hierarchical Inheritance

```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```

Example

```
#include <iostream>
using namespace std;
class Shape
{
    public:
    int a;
    int b;
    void get_data(int n,int m)
    {
        a= n;
        b = m;
    }
};
class Rectangle : public Shape
{
    public:
    int rect_area()
    {
        int result = a*b;
        return result;
    }
};
class Triangle : public Shape
{
    public:
    int triangle_area()
    {
        float result = 0.5*a*b;
        return result;
    }
};
int main()
{
    Rectangle r;
    Triangle t;
    int length,breadth,base,height;
cout << "Enter the length and breadth of a rectangle: " << "\n";
    cin>>length>>breadth;
    r.get_data(length,breadth);
    int m = r.rect_area();
    cout << "Area of the rectangle is : " <<m<< endl;
 cout << "Enter the base and height of the triangle: " << endl;
    cin>>base>>height;
    t.get_data(base,height);
    float n = t.triangle_area();
cout <<"Area of the triangle is : " << n<<endl;
    return 0;
}
```

Output of previous program

Output:

```
Enter the length and breadth of a rectangle:
23
20
Area of the rectangle is : 460
Enter the base and height of the triangle:
2
5
Area of the triangle is : 5
```